Director's Research Guide

What we built, what we tried, what happened, and what to do next
(driving models that read a bird's-eye view, tested in the MetaDrive simulator)

Prepared for the project director
Repository state of 12 November 2026

Contents

1 The project on one page **2**

1.1 What this document is . . . 2

1.2 The question, and how it changed . . . 2

2 How a driver works: the ideas in plain language **3**

2.1 What the driver sees . . . 4

2.2 Three ways of teaching a driver . . . 4

3 The models we trained and where they are **5**

4 How we tested **6**

5 Results **6**

5.1 The main result: 200 drives in clear weather . . . 6

5.2 The image-only driver . . . 8

5.3 Does weather make driving worse? . . . 9

5.4 Caution-aware driver against the proposed driver, 200 drives in each weather . . . 10

5.5 Three side studies . . . 11

6 Reinforcement learning: the four attempts **12**

7 Where the drivers fail, and why **15**

8 Videos **16**

9 Comparison with the current GitHub repository **17**

10 What we can claim, and what we cannot **18**

11 What to do next **18**

12 Where to find everything **19**

1 The project on one page

What we built and what we found

We built software "drivers" for a car in a driving simulator. Each driver looks at a small top-down picture of its surroundings and a few numbers about the car, and decides how to steer and how much to accelerate or brake. We compared ways of *teaching* such a driver, measured how fog and rain damage the camera system that would feed it, and tested whether that damage makes the drivers worse.

- **Best driver:** the *Proposed Bird's-Eye-View Driving Model* succeeds in **92.5%** of 200 test drives. The two baseline drivers that are on GitHub succeed in **85.0%** and **74.5%**. It ties the rule-based teacher it learned from (95.0%).
- **Weather:** simulated fog and rain make the object detector miss most vehicles (it finds 72% in clear weather and 1.4% in the heaviest rain), yet driving success in sparse traffic stays at **89–91%**. The original idea, a driver that adapts to weather, has nothing to adapt to here.
- **Reinforcement learning** (teaching by trial and error with a score) was tried four times and **never improved a driver**; the last attempt made it worse in every weather condition.
- **Failures:** most of the remaining failures are crashes in the first seconds that a better-informed teacher avoids: the driver's input does not show enough.
- **GitHub:** it holds the teammates' modules and the two baseline drivers only. All other work and every finding in this document are on this computer only.

1.1 What this document is

This is the *Director's Research Guide*: about twenty pages that explain what we built, what we tried, what happened, why, and what to do next, without needing to know the code. Every model is called by its descriptive name; the internal nicknames of the old logs appear nowhere except in the table of Section 3 (once) and the run labels C2, Probe C and C3 of reinforcement-learning attempt 1, which are part of their file names. The *Complete Technical Research Report* contains every detail, every file and every number behind these pages; each section below says where to find it.

1.2 The question, and how it changed

The research plan asked: *can a driving policy keep following its route, and use less energy, when its camera perception is degraded by fog and rain?* Answering it needed four pieces of work, and each one produced a finding:

Piece of work	Finding
1. A driving model that works	Teaching by repeated drive-and-correct (dataset aggregation) gives a driver that beats the baselines and ties its teacher.
2. A measurement of the weather damage	Fog and rain remove most vehicle detections but do not make the distance estimates worse for the vehicles that are found.
3. A test of whether the damage matters	It does not lower success in sparse traffic, so weather adaptation cannot be shown here.
4. Reinforcement learning, including an energy-aware reward	Four attempts; none improved a driver; the reward let the driver trade safety for speed.

The research value is therefore in the evaluation, the measurement, the failure analysis, the negative

results, and the directions they point to (Section 11).

2 How a driver works: the ideas in plain language

Table 1: The key terms of this project. The Complete Technical Research Report has a full dictionary of every technical term.

Term	Meaning, and why it matters here
Driving model (policy)	A function that turns what the car sees into two commands: steering, and throttle or brake. Every model in the project is one.
MetaDrive simulator	The driving simulator. Each test is an *episode* on a *scenario seed*: a number that fixes the road, the route and the other cars, so every model can be tested on identical scenarios.
Bird's-eye-view grid	The driver's main input: a 64 by 64 top-down picture around the car, 20 m by 20 m. Each cell says free, unknown, our car, or a detected vehicle.
Map overlay	The route and the road edges, drawn into the grid from the map. It tells the driver where the road is.
Vehicle-state numbers	Speed, route progress, distance to the goal, and (in some models) lateral offset, heading error and road curvature.
Behaviour cloning	Teaching a driver by showing it a handful of recorded good drives. Cheap, but after one small mistake the driver is in a situation it never saw and cannot recover.
Dataset aggregation	Teaching by repetition: the driver drives, a teacher says what it should have done in the situations the driver actually reached, and these corrections are added to the lessons. Fixes the recovery problem.
Teacher and "information gap"	A teacher is a rule-based driver that sees the exact state of the simulator. The student sees only the grid, so there are decisions it cannot copy because it cannot see why.
Reinforcement learning	Teaching by trial and error: the driver gets a score (the *reward*) after every step and is changed to score more.
Object detector and stereo depth	The two camera steps that put a vehicle into the grid: one finds it in the picture, the other estimates how far away it is.
Success rate and confidence interval	The share of test drives that reach the destination, with the range of values compatible with that count. Two models are compared on the *same* drives (a *paired* test), which is far more sensitive.
Pilot, reported, diagnostic	Pilot: 10–40 drives, a hint. Reported: 200 paired drives, a claim. Diagnostic: one or two replayed drives that explain a mechanism.

2.1 What the driver sees

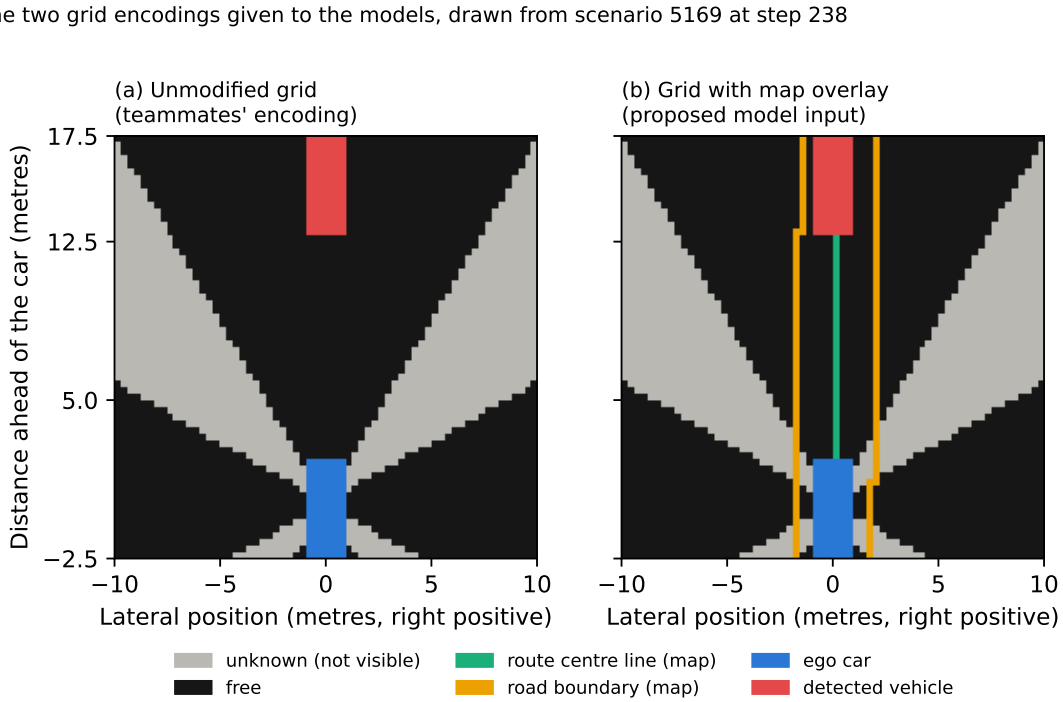


Figure 1: The two input encodings of the bird's-eye-view grid, drawn from one real scenario. The model's input is a 64 by 64 grid covering 10 metres to each side, 17.5 metres ahead and 2.5 metres behind the car. Left: the teammates' encoding (unknown, free, the ego car and detected vehicles). Right: the same grid with the route centre line and road boundaries from the map added. Both are drawn from scenario seed 5169 at step 238, with the same vehicle in view.

Plain-English reading. In the left grid the car sees free space and a vehicle ahead but nothing about the road. In the right grid it also sees where the lane edges and the planned route are.

Research implication. The map overlay gives the model the road geometry directly, which is why three vehicle-state numbers (lateral offset, heading error, curvature) could be removed from its numeric inputs. It also creates the dependence on an accurate position estimate studied above.

2.2 Three ways of teaching a driver

1. **Copy a few recordings** (behaviour cloning). The two baseline drivers on GitHub were taught this way from five recordings.
2. **Let it drive and correct it** (dataset aggregation). This produced the Proposed Bird's-Eye-View Driving Model.
3. **Trial and error with a score** (reinforcement learning). Section 6.

The teacher in 1 and 2 is a rule-based driver. Two were used: MetaDrive's *Stock Intelligent Driver Model* (keeps its lane and follows the car ahead) and a *Collision-Aware Teacher* that adds predictive braking when a collision is foreseen within about 2.4 seconds.

3 The models we trained and where they are

In plain words

Thirty-three model files were saved, all in the folder `trained_models/` with descriptive file names (for example `Proposed_BEV_Driving_Model.pt`). Only the two baseline drivers are also on GitHub. The table names each model, says what question it was built to answer, and gives the internal nickname that appears in old logs, once.

Table 2: The models, the question each one answers, and where it is.

Model (descriptive name)	Question it answers	Old nickname	On GitHub
Baseline Driving Model Trained Without Traffic	What does the teammates' behaviour cloning achieve from five recordings with no other vehicles?	repo_bc_clear	yes
Baseline Driving Model Trained With Traffic	Same, with traffic in the recordings (the repository's recommended start).	repo_bc_sparse	yes
Proposed Bird's-Eye-View Driving Model	How good is a driver taught by repeated drive-and-correct, shown the grid with the map overlay and three vehicle-state numbers?	ov_split	no
Five input-design variants (plain grid or overlay; six, three or no vehicle-state numbers; or numbers only)	Which input matters?	base_a, ov_both, ov_bev_only, arch_bev_only, arch_scalar_only	no
Proposed Model Trained From the Collision-Aware Teacher	Does a better teacher give a better driver? (not yet tested on 200 drives)	sparse_overlay_split_safe_teacher	no
Weather-Degradation-Trained Driving Model	Does training under the measured perception damage help?	deg_base	no
Caution-Study Caution-Aware Model and its Control (caution fixed to zero)	Should a driver be more careful when its perception is less reliable?	caut_a, ctrl_nocaut	no
Four Position-Error-Study Models	Can the map-overlay driver survive a wrong position estimate?	pose_aug_*, pose_cam_*	no
Reinforcement-learning models (attempts 1 to 4; 12 files)	Can trial and error improve a driver?	C2, probe_C, C3, pilot_cond, rl_*, ppo_plain, ppo_ada	no
Image-Only Driving Model (Front Camera)	Can a driver read the camera picture directly, without the grid?	vcam	no
Early Camera-View Version 2 Models (3 files)	Superseded early experiment, not used for results.	bcv2_*	no
YOLO Version 8 Nano Object Detector Weights	The third-party detector used by the perception pipeline.	yolov8n	no (local copy)

Where the models are

- **All 33 models:** `D:/bev_Fresh/bevcapstone/trained_models/`, in six folders named by purpose. Every file is a byte-identical copy of the original checkpoint (hash verified). `MODEL_NAME_MAPPING.md` gives each model's previous file name; `MODEL_MANIFEST.json` gives hashes and descriptions.

- **On GitHub:** the two baseline files in `bev_policy/checkpoints/`. Nothing else.

4 How we tested

In plain words

Every model drove the *same 200 scenarios* that nobody had used for training or for choosing models. A drive counts as a success only if it reaches the destination without a collision and without leaving the road. Because every model drives the same scenarios, two models can be compared on the drives where they differ, which detects a difference of 7.5 percentage points with 200 drives.

- **Simulator and scenarios.** MetaDrive 0.4.3; a scenario seed fixes the road, route and traffic; traffic is sparse (density 0.05) for all main results and dense (0.2) only for diagnostics; 800 steps of 0.1 s = 80 s per drive.
- **Separate scenario blocks:** training seeds 2000–2999; validation seeds 3100–3139 (to choose models); test seeds 1010–1049 (40 drives, pilot) and 5000–5199 (200 drives, the main test).
- **Four ways a drive ends:** success, collision, left the road, timeout.
- **Statistics:** 95% Wilson confidence intervals; exact McNemar test and bootstrap interval for paired differences.
- **What the drivers see during the tests.** The full camera pipeline costs about two seconds per step, so the drivers were tested in a *camera-free surrogate*: the grid is built from the simulator's list of vehicles (an ideal detector), then degraded by the measured weather effect where weather is studied. This departs from a rule written in the GitHub README (Section 9); results are valid for the driving models, not for the camera system as a whole.

5 Results

5.1 The main result: 200 drives in clear weather

In plain words

The Proposed Bird's-Eye-View Driving Model succeeds in 92.5% of 200 drives, against 85.0% and 74.5% for the two baselines, and 95.0% for the rule-based Collision-Aware Teacher it learned from. The proposed driver and its teacher cannot be told apart statistically. The image-only driver, which reads the camera picture instead of the grid, succeeds in only 0.130 of the drives.

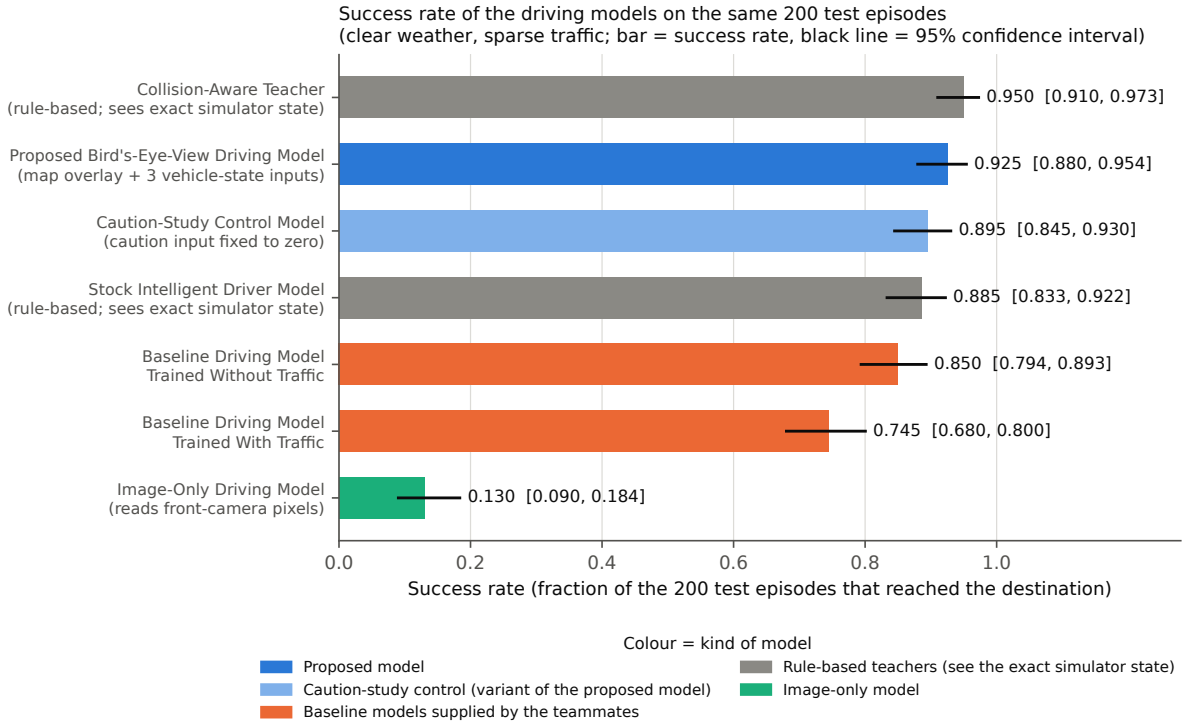


Figure 2: Success rate of the driving models on the same 200 test episodes. Each bar is the share of 200 identical test episodes (scenario seeds 5000 to 5199, clear weather, sparse traffic) in which a model reached the destination without a collision and without leaving the road. The black line is the 95% Wilson confidence interval, the range of success rates that is statistically compatible with the observed count. Grey bars are rule-based teachers that see the exact simulator state; orange bars are the two baseline models supplied by the teammates; blue bars are models trained in this project.

Plain-English reading. The Proposed Bird's-Eye-View Driving Model succeeds in 92.5% of the episodes. The two baseline models succeed in 85.0% (trained without traffic) and 74.5% (trained with traffic). The proposed model is about as good as the Collision-Aware Teacher (95.0%) whose behaviour it learned; the intervals overlap, so the two cannot be told apart. The Image-Only Driving Model, which reads camera pixels instead of the bird's-eye-view grid, reaches 0.130 (interval 0.090 to 0.184), far below every model that uses the grid.

Research implication. Training a model by letting it drive and asking a teacher what it should have done (dataset aggregation) produces a model that matches its teacher and clearly beats few-demonstration behaviour cloning. This figure does not say *which part* of the proposed design (the training method or the map overlay) causes the gain; that needs the additional evaluations listed in the paper-directions section.

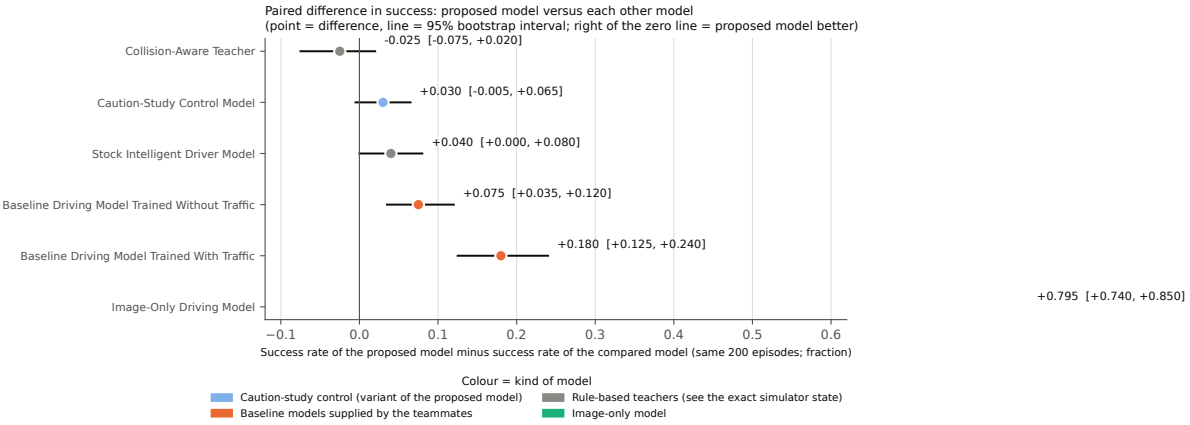


Figure 3: Paired difference in success between the proposed model and each other model. For each comparison the difference (proposed model minus the other model) is computed on the same 200 episodes. The point is the difference; the black line is the 95% bootstrap interval obtained by resampling episodes 5000 times. A line entirely to the right of the zero line means the proposed model is better with statistical support.

Plain-English reading. The proposed model is better than the baseline trained without traffic by 0.075 (interval +0.035 to +0.120) and better than the baseline trained with traffic by 0.180 (interval +0.125 to +0.240). Its differences to the Collision-Aware Teacher (−0.025), the caution-study control (+0.030) and the stock driver model (+0.040) have intervals that include or touch zero. The Image-Only Driving Model is far below it.

Research implication. The claim that the proposed model beats the two baselines is statistically supported. No claim is made that it beats the teachers or the control. Using paired comparisons (same scenarios for every model) is what makes a difference of 0.075 detectable with 200 episodes.

What the main result does not show

It does not separate *why* the proposed driver is better: three things changed together (repeated drive-and-correct teaching instead of five recordings; the map overlay; the choice of three vehicle-state numbers). The plain-grid variant with six numbers was tested on only 10 and 40 drives (0.70 and 0.875 against 0.80 and 0.925), not on 200. Nothing is claimed about dense traffic, where even the teachers fail more than half of the drives. The results are for the camera-free surrogate.

5.2 The image-only driver

The Image-Only Driving Model sees only a front-camera picture (128 by 96 pixels): no grid, no numbers. On the same 200 drives it succeeds in 0.130 (interval 0.090 to 0.184); 0.200 of the drives end in a collision, 0.605 by leaving the road and 0.065 by timeout. Its mean speed is 9.20 metres per second and its estimated energy use 276.6 watt-hours per kilometre.

Research implication

Reading the picture directly is far harder than reading the grid. The grid (a compact, structured summary of the surroundings, produced by the detector and the stereo depth) is what makes the proposed driver possible at this amount of training and data. Whether a camera network trained with far more data could close the gap is not answered here.

5.3 Does weather make driving worse?

In plain words

First we measured the damage. In clear weather the object detector finds 72% of the vehicles in view; in the strongest fog 31%; in rain of intensity 0.6 only 15%; in the heaviest rain 1.4%. The distance estimate for vehicles that *are* found does not get worse, so the damage is almost entirely *missed vehicles*. Then we tested the drivers with that damage applied. **Driving success did not fall:** 89–91% for the proposed driver in all four weather conditions. A pre-registered hypothesis that the damage would hurt a driver trained without it was therefore *not* confirmed.

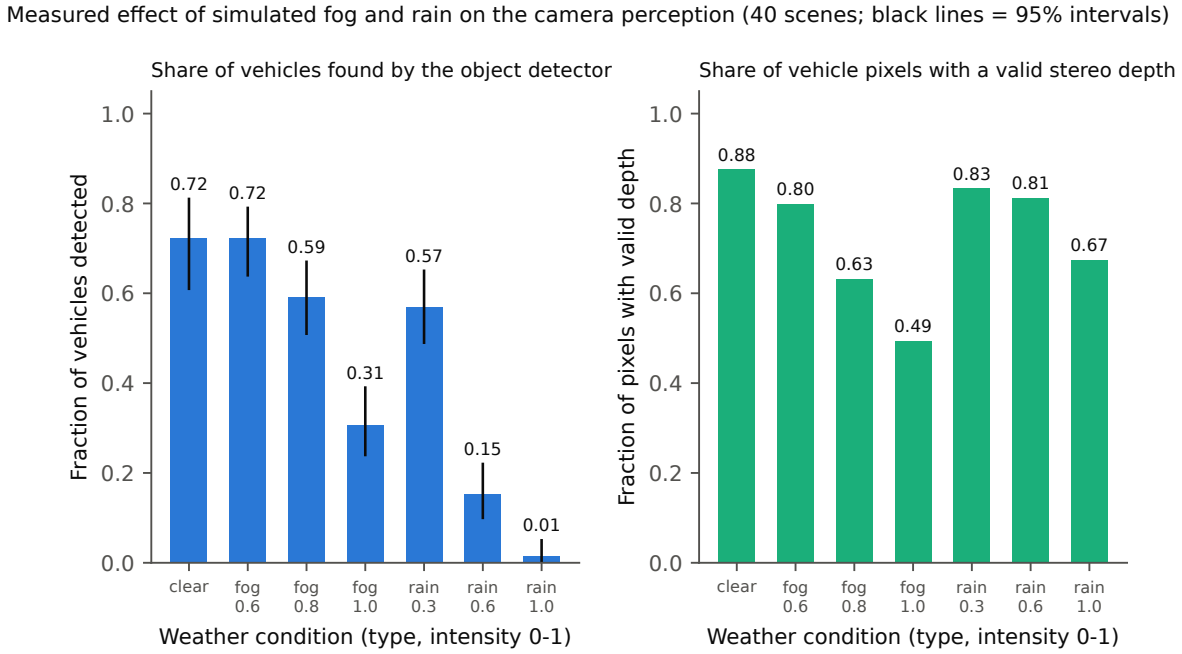


Figure 4: Measured effect of simulated fog and rain on the camera perception. Left: the share of vehicles in view that the YOLO object detector finds, per weather condition (vertical lines are 95% intervals). Right: the share of vehicle pixels at which the stereo-depth algorithm returns a valid distance. Weather conditions are the teammates' image augmentation at the intensity shown (0 = none, 1 = strongest). 40 scenes; 72 vehicle observations in clear weather and 144 in each weather condition.

Plain-English reading. In clear weather the detector finds 72% of the vehicles. In fog of intensity 1.0 it finds 31%, in rain of intensity 0.6 it finds 15% and in rain of intensity 1.0 only 1.4%. Stereo depth stays valid for 88% of pixels in clear weather and 49% in the strongest fog, but remains at 67% to 83% in rain.

Research implication. The perception measurement confirms that the simulated weather is a real degradation of the camera pipeline, mostly through *missed vehicles*. For vehicles that are detected, the distance estimate is not worse. This tells us what a driving model would have to cope with, and explains why the driving results in the next figures are unaffected.

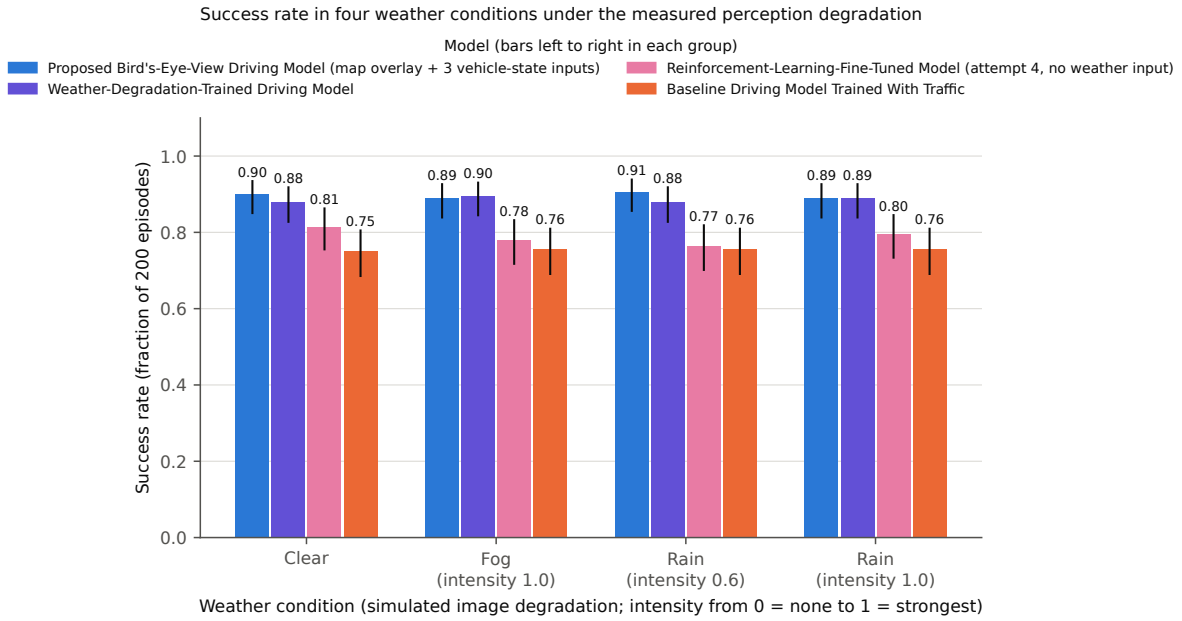


Figure 5: Success rate in four weather conditions under the measured perception degradation. Four models were each evaluated on the same 200 episodes in clear weather, fog of intensity 1.0, rain of intensity 0.6 and rain of intensity 1.0. In every weather condition the input of the model was degraded as measured: vehicles can disappear from the bird's-eye-view grid, or be placed at a wrong distance. Black lines are 95% intervals.

Plain-English reading. The proposed model stays between 0.89 and 0.91 in all four conditions; the weather-degradation-trained model between 0.88 and 0.90. The reinforcement-learning-fine-tuned model is lower in every condition (0.765 to 0.815) and the baseline trained with traffic is flat at about 0.75.

Research implication. Under the perception degradation that was measured and implemented, weather does not lower driving success in sparse traffic. It follows that a robustness or weather-adaptation claim cannot be made from this experiment, and that a weather-adaptive method would have nothing to adapt to here. The next research step is to find the conditions (denser traffic, harsher loss of vehicles) under which weather begins to matter.

What this means for the original question

The question "can the driver adapt to degraded perception?" cannot be answered here, because the degradation does not hurt sparse-traffic driving. The research step that follows is to find the conditions in which weather *does* start to matter (denser traffic, more severe loss of vehicles), which is Direction 2 in Section 11.

5.4 Caution-aware driver against the proposed driver, 200 drives in each weather

On 200 drives per condition the caution-aware driver matches the proposed driver in clear weather (0.890 against 0.925, not significant), fog (0.895 against 0.895) and rain 0.6 (0.890 against 0.900), and is lower in the strongest rain (0.820 against 0.900, $p = 0.033$, all extra failures are timeouts). In weather it uses about 35% less estimated energy (59.7 to 62.7 against 93.0 to 96.5 watt-hours per kilometre). All the extra failures are timeouts: with a 160-second limit instead of 80 seconds (a post-hoc check) the caution-aware driver finishes 0.940, 0.955 and 0.935 in fog 1.0, rain 0.6 and rain 1.0, a tie with the proposed driver (no difference is significant) with fewer collisions. Full table, paired tests and figure: Complete Technical Research Report, section "Caution-aware model against the proposed model and both baselines".

5.5 Three side studies

Caution when perception is unreliable (40 drives, pilot). A driver given a "caution" number derived from the measured detector reliability slowed from about 8.0 to 5.6–6.2 metres per second in weather and used about 30% less estimated energy (84.7 to 56–59 watt-hours per kilometre), with more timeouts (7.5–15%) and no gain in success. This is *learned adaptive behaviour*: the driver was taught by a teacher whose speed and gap depend on caution, and reproduces that dependence by itself, while the control with caution fixed at zero does not slow down. The benefit is not shown (success did not rise; weather did not hurt the proposed driver anyway), and the caution value is supplied from the known weather rather than sensed by the driver.

Which input matters (10 drives, pilot). The map overlay with the three vehicle-state numbers was nominally the best design (0.80 in sparse and 0.60 in dense traffic); the overlay with no numbers was the worst (0.30 and 0.10). Intervals are very wide.

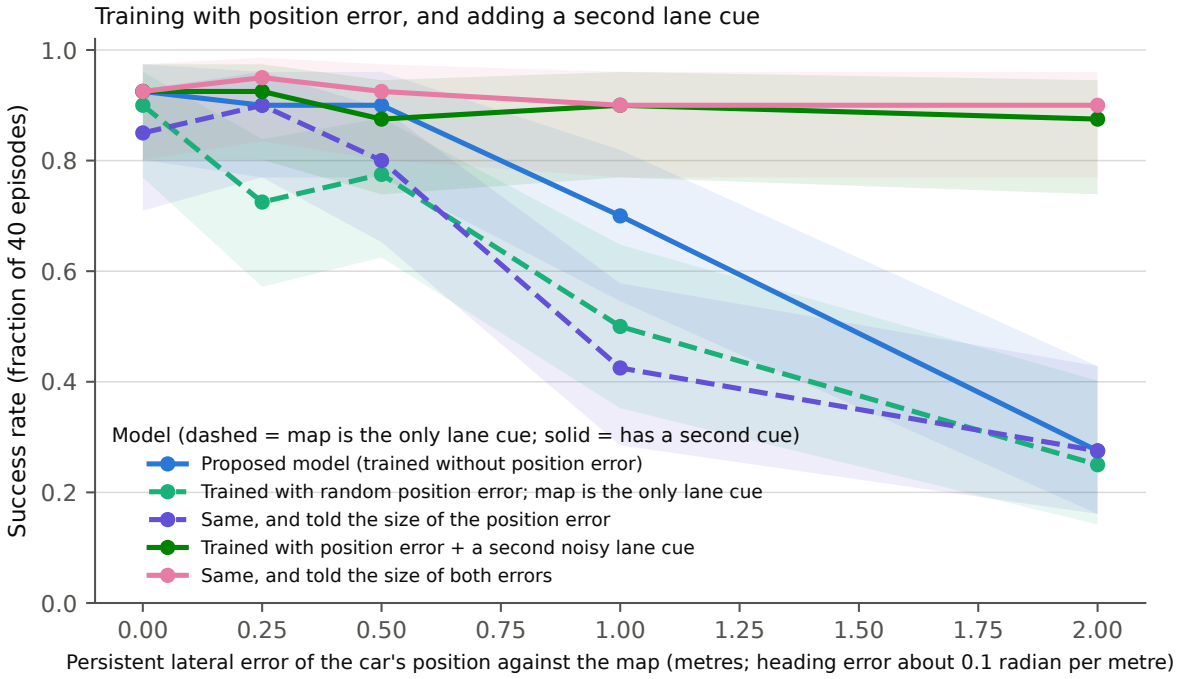


Figure 6: Training with position error, and adding a second lane cue. The proposed model reads the road from a map drawn into its input, so it needs to know where the car is on the map. Here the position estimate is shifted by a constant lateral distance (horizontal axis, metres; a small heading error grows with it). Success over 40 episodes for the proposed model and four models trained with random position errors; bands are 95% intervals. Dashed lines: the map is the only source of lane position. Solid green and pink lines: a second, independent and noisy lane measurement (lateral offset and heading) is also given to the model. 40 episodes per point.

A wrong position estimate (40 drives, pilot).

Plain-English reading. Training with position errors alone does not help (dashed lines, 0.25 to 0.28 at 2 metres of error). Adding a second independent lane cue keeps success between 0.875 and 0.95 for all error sizes up to 2 metres (solid lines).

Research implication. Robustness to a wrong position estimate comes from *redundancy* (two sources that can disagree), not from noise augmentation. Caution: the second cue is simulated (simulator truth plus noise), so the result shows that the network can exploit redundancy, not that a real lane detector would be accurate enough.

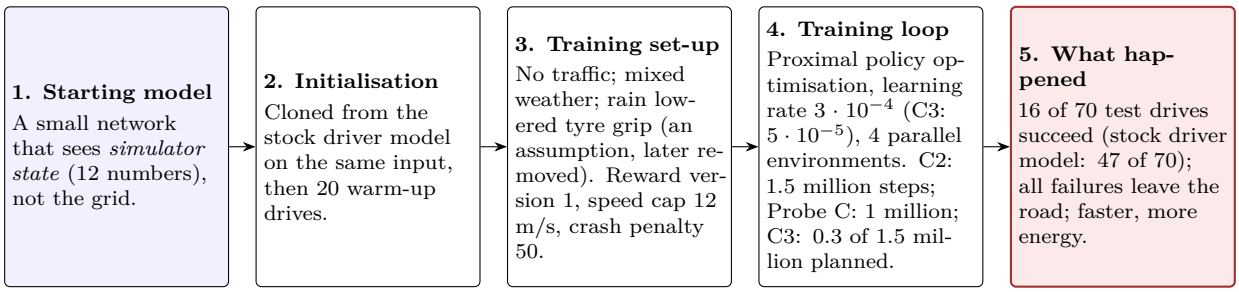
6 Reinforcement learning: the four attempts

In plain words

Reinforcement learning teaches a driver by trial and error: it drives, a formula (the *reward*) scores each step, and the driver is changed to score more. We hoped it would improve the imitation-trained drivers, make them energy-aware and make them weather-adaptive. We tried four times with different starting models and rewards. **No attempt produced a better driver, and the last one, the only one tested on 200 drives, made the driver worse in every weather condition.** Each attempt below is drawn as a flow from the starting model to what happened, followed by the reward, the settings, the number of training steps, the evaluation, why it failed and what we learned.

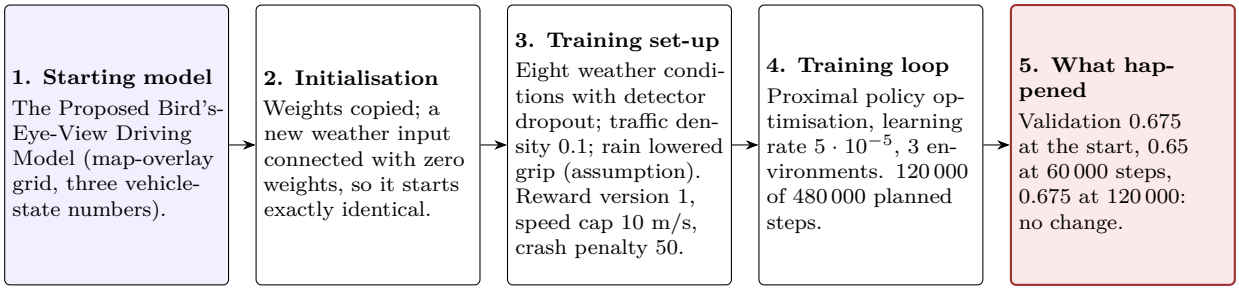
The reward in words. Version 1 (attempts 1–3) pays for forward progress and subtracts for lateral error, heading error, jerky control, braking, speeding, standing still and for crashes (penalty 50; success +20). Version 3 (attempt 4) pays for progress, subtracts for lateral and heading error, for a predicted collision within 3 seconds, for being closer than 6 metres to the car ahead, for energy used and for jerky control, has *no* braking penalty, and pays +50 for success and −100 for a crash. The full formulas are in the technical report.

Attempt 1: a "privileged teacher" trained by reinforcement learning (runs C2, Probe C, C3)



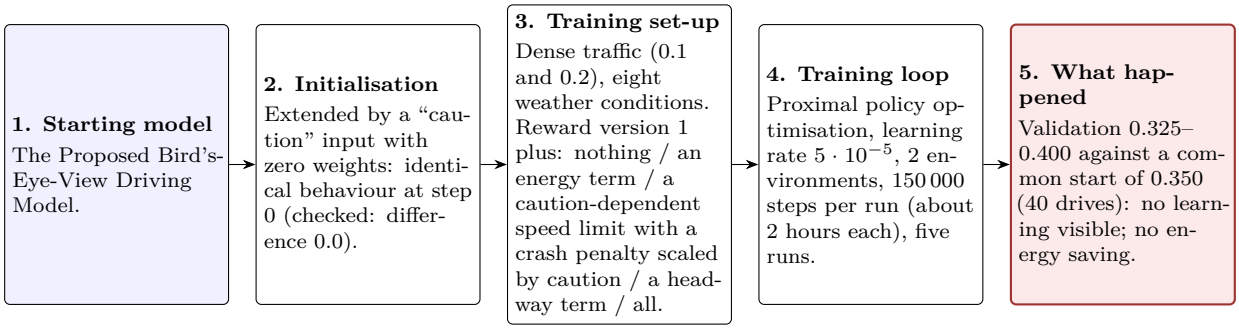
Why it failed	Reward kept rising (C2 +9%, Probe C +41%) while success did not (0.59 → 0.45; 0.62 → 0.60) and the driver became much faster (10.4–10.7 against 7.9 m/s): consistent with the reward paying more for speed than it costs to leave the road occasionally (a hypothesis; no ablation).
What we learned	A rising reward is not evidence of better driving; a teacher worse than the rule-based one gives nothing to copy, so the project moved to rule-based teachers.

Attempt 2: fine-tuning the Proposed Bird's-Eye-View Driving Model with a weather input



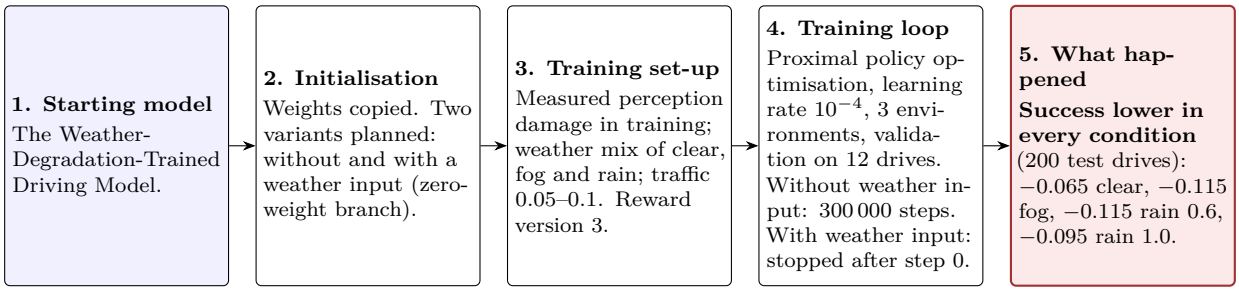
Why it did not help	The driver was already close to what its input allows, and the grip coupling confounded the effect of weather. Not separated experimentally.
What we learned	A weather input cannot help a driver that weather does not hurt; the grip coupling was removed from later work.

Attempt 3: five reward combinations from the proposed model



Why it did not help	With 40 drives one drive is 0.025 and the interval of a value is about ± 0.15. The failures come from crossing and merging vehicles that the grid shows late; trial and error cannot learn to avoid what the driver cannot see.
What we learned	40 validation drives cannot choose a model; a reward cannot add missing information; an energy term needs a speed-matched control.

Attempt 4: reward version 3 under the measured perception damage



Why it failed	Collisions roughly doubled (0.20 against 0.11 in rain 1.0), speed rose from about 7.9 to 8.8 m/s and energy from about 86 to 103 watt-hours per kilometre: the same signature as attempt 1. Hypothesis: a crash penalty of 100 is small compared with the progress earned over hundreds of steps.
What we learned	Validation on 12 drives at 1.0 hid the loss; the weather-input run did not run, so the pre-registered adaptation test could not be done.

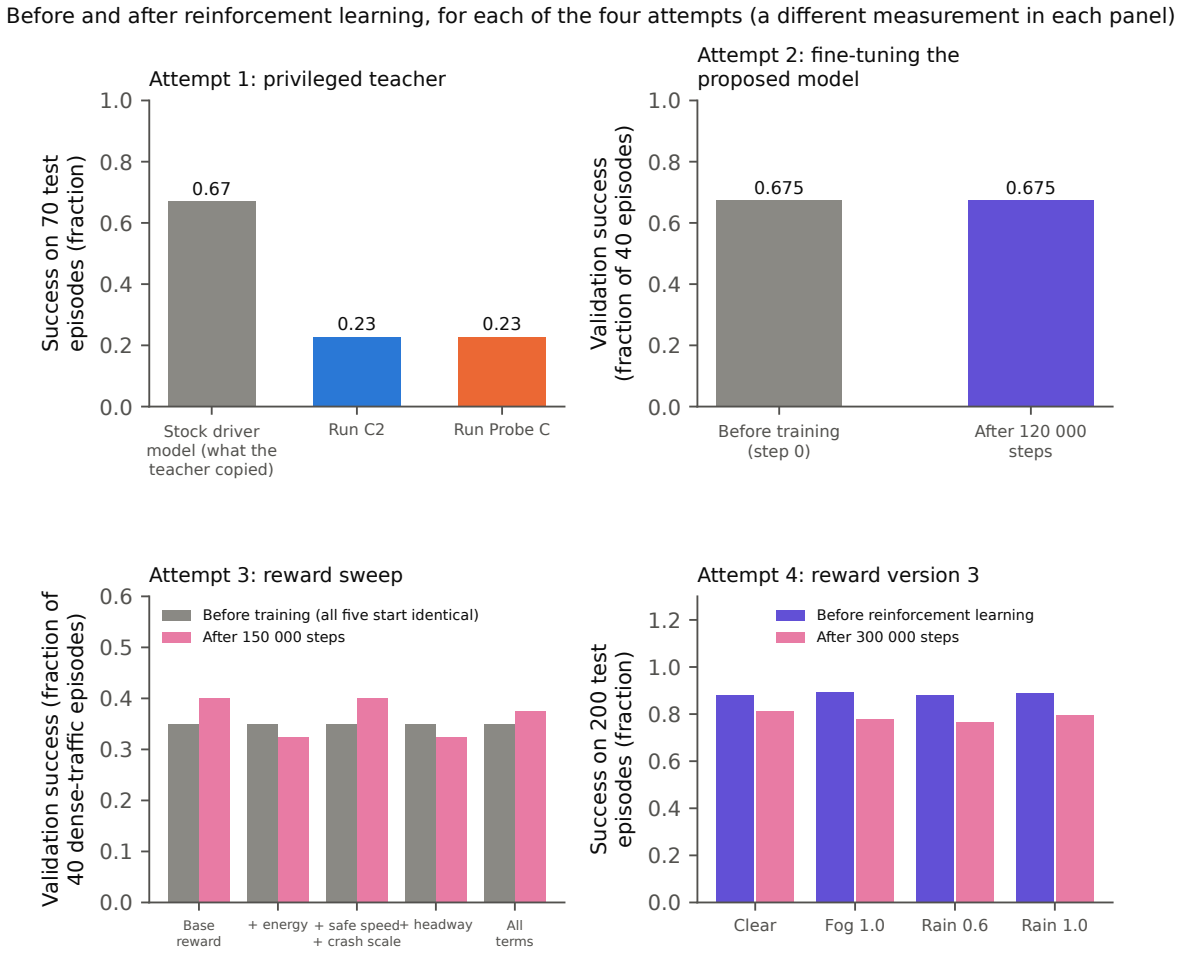


Figure 7: Before and after reinforcement learning, for each of the four attempts. Each panel compares the model before reinforcement learning with the model after it, using the best available measurement for that attempt: 70 test episodes (attempt 1), 40 validation episodes (attempts 2 and 3) and 200 test episodes per weather condition (attempt 4).

Plain-English reading. Attempt 1: the trained teachers succeed in 23% of 70 test episodes, against 67% for the stock driver model they started from. Attempt 2: unchanged (0.675). Attempt 3: changes of -0.025 to $+0.05$, within the noise of 40 episodes. Attempt 4: success falls in every weather condition, by 0.065 to 0.115.

Research implication. Reinforcement learning never improved a model in this project and twice made it worse. The reasons (the reward lets the policy trade safety for speed; the evaluation was too small; the starting model is already close to what its input allows) are analysed in the reinforcement-learning section.

Research implication of the four attempts

Reinforcement learning is a documented *negative result with a mechanism*: the reward let the driver trade safety for speed, the starting driver was already near what its input allows, and the evaluation during training was too small to notice. A fair second attempt would start from the Collision-Aware-Teacher student, validate each candidate on at least 200 paired drives, use a crash penalty large enough to dominate speed gains, constrain the new policy to stay near the old one, and give the driver the information it lacks.

7 Where the drivers fail, and why

In plain words

The proposed driver fails 15 of 200 drives: 12 collisions, 2 times leaving the road, 1 timeout. The Collision-Aware Teacher succeeds in **14 of those 15**, so the failures are not unsolvable scenes: they are things the student cannot copy. Eight of the 15 end within the first 12 seconds, and in seven of them every other model also collides; only the collision-aware teacher survives.

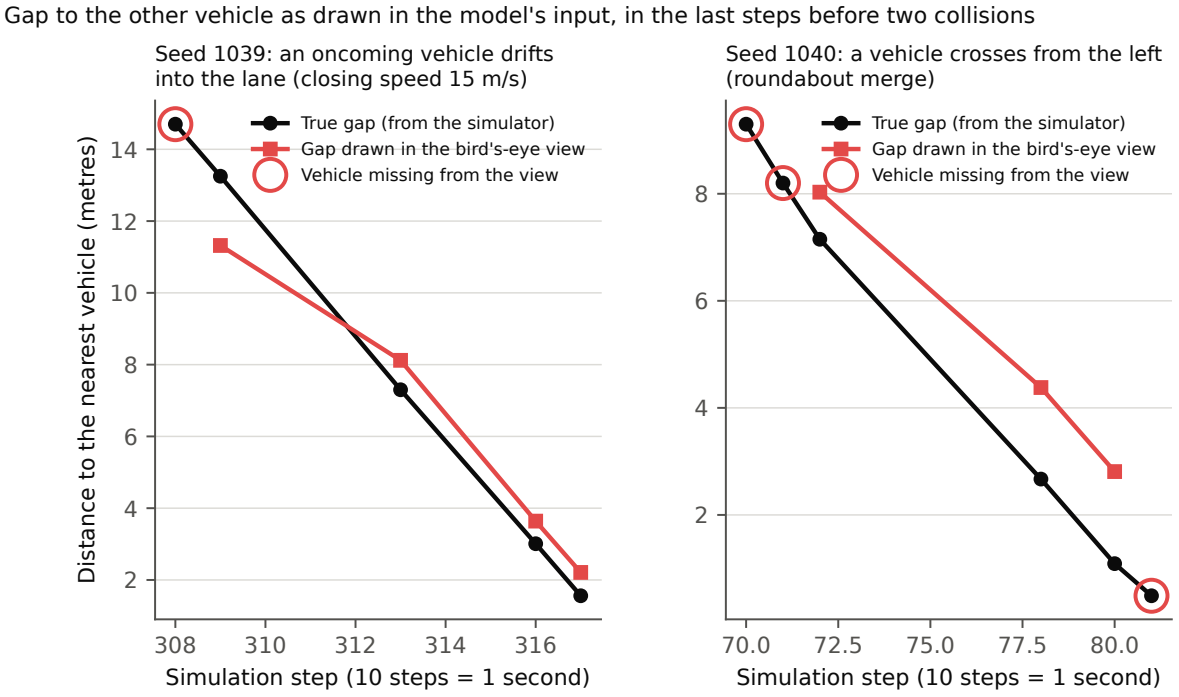


Figure 8: Gap to the other vehicle as drawn in the model's input, in the last steps before two collisions. For two crashes of a bird's-eye-view model, the true distance between the car and the nearest vehicle (black) is compared with the distance that the 64 by 64 grid shows (red). A hollow circle marks steps at which the vehicle is not in the grid at all. These numbers are copied from the diagnosis note of 10 November (two replayed episodes).

Plain-English reading. The oncoming vehicle in the left panel first appears in the grid only 0.9 seconds before impact. In the right panel the grid shows the crossing vehicle 1.7 metres farther away than it really is, and the vehicle disappears from the grid 0.1 seconds before the collision.

Research implication. The model's view of the other vehicles is late and coarse: vehicles are drawn as blocks without heading, and the grid ends 17.5 metres ahead. This is a limit of the representation that every model using this grid shares, and it is the leading explanation for the remaining failures.

- **The driver's view is late and coarse.** Vehicles are drawn as blocks without heading and the grid ends 17.5 m ahead, while the teacher sees every vehicle up to 45 m with exact velocities. This is an *information gap*; it is in the teammates' own grid code on GitHub (`bev_state/assembler.py`), so every model using the grid, including the two baselines, shares it.
- **Dense traffic is unsolved**, even by the rule-based teachers (success 40–45%), so no driver trained from them can do better; the main results are restricted to sparse traffic.
- **The baselines fail differently:** the traffic-trained baseline leaves the road in 13.5% of the

drives (the proposed driver: 1.0%), the signature of drifting after a small error with only five recordings to learn from.

- **The map overlay has a price:** a constant error of 1 m in the car's estimated position lowers the proposed driver's success from 0.925 to 0.70, and 2 m to 0.275; a second, independent lane measurement repairs it (0.875–0.95), so redundancy, not noise training, is what protects it.

8 Videos

In plain words

One scenario (number 5169) was recorded in four weather conditions with the front camera only, in smooth real time, for the two baselines, the proposed driver and the image-only driver. Both baselines collide in all four conditions; the proposed driver succeeds in all four. The image-only driver fails in all four (see the table in the technical report). Files are in `report/videos/`.

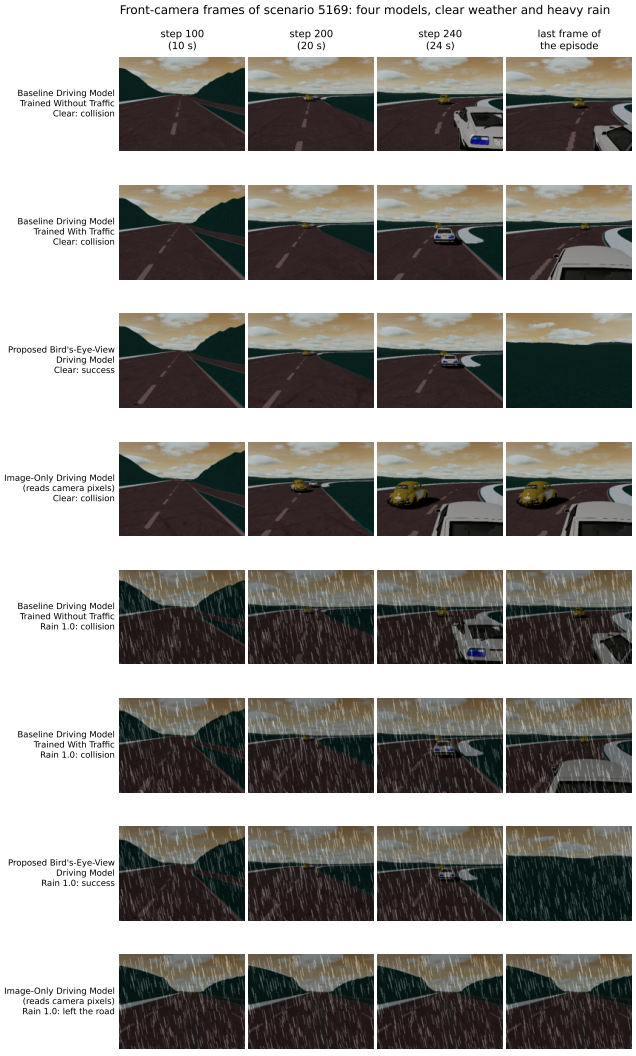


Figure 9: Frames of the front-camera videos of scenario 5169. Each row is one model in one weather condition (clear, or rain of intensity 1.0); the columns are moments of the episode (steps 100, 200, 240 and the last frame; 10 steps = 1 second). The picture is the front-left camera of the teammates' rig with the weather augmentation applied. Videos for fog of intensity 1.0 and rain of intensity 0.6 are in the same folder.

Plain-English reading. Both baselines collide with another vehicle at about step 250 (25 seconds). The proposed model does not collide and drives on until it reaches the destination at about step 700 (70 seconds). The Image-Only Driving Model collides at step 230 in clear weather and leaves the road after 88 steps in the heaviest rain. Heavy rain makes the picture much worse but does not change what the grid-based models do.

Research implication. This scenario was chosen *because* the baselines fail on it, so it illustrates a typical failure but says nothing about how often it happens (the 200-episode results do). The grid-based models do not read these pixels; they read the simulator-built grid with weather applied as measured. Only the image-only model reads pixels, and it fails here in all four weather conditions.

9 Comparison with the current GitHub repository

In plain words

GitHub (checked 12 November 2026; newest commit of 8 September 2026) contains the teammates' modules and exactly two model files, the two baselines. Everything produced in this project is on this computer only.

Component	On GitHub	In this project (local only)
Models	Two baseline checkpoints	33 models, including the proposed driver and four groups of reinforcement-learning models
Training	Behaviour cloning from five recordings	Dataset aggregation, two teachers, caution, position-error and degraded-perception variants
Grid	Plain grid; map overlay "unimplemented"	Map overlay as a policy-side wrapper (the grid code is unmodified)
Weather	"Preliminary tooling", no measurement	Measured detector and depth degradation; 200-drive test
Reinforcement learning	"Missing"; the traffic-trained baseline is recommended as the start	Four attempts, none from that checkpoint; all negative
Baseline success	0.90 (9 of 10 drives)	0.745 on 200 drives (interval 0.680 to 0.800)

Departures from rules written on GitHub

- The README says not to feed exact simulator vehicle positions to a learned model. The camera-free surrogate does (an ideal detector), degraded statistically where weather is studied.
- The baselines were evaluated through that surrogate, not through the repository's evaluator.
- Three teammate files carry local edits (an optional extra-input branch; defaults unchanged).
- The first two reinforcement-learning attempts coupled rain to tyre grip, against the README's statement that weather does not change dynamics (later removed).

10 What we can claim, and what we cannot

We can claim (200 paired drives)	We cannot claim
The proposed driver beats both baselines and ties its teacher in sparse traffic.	That the map overlay (rather than the training method) causes the gain: untested at 200 drives.
Simulated fog and rain remove most detections but do not lower sparse-traffic success.	Any robustness or weather adaptation, or any result for physical rain or fog.
Four reinforcement-learning attempts failed, one significantly (mechanism hypothesised).	That reinforcement learning cannot work: only that these attempts did not.
Most remaining failures are imitation failures solved by a better-informed teacher.	Results for dense traffic, for the live camera pipeline, or between training seeds (one seed per model).

11 What to do next

In plain words

Seven possible papers exist. The best *single* paper combines the first three: a thorough evaluation of training strategies (imitation against reinforcement learning), the measured weather model with its null result, and the information gap that limits the drivers.

#	Direction	Evidence in hand, and what is missing	Cost
1	Pitfalls of closed-loop evaluation	0.975 on 40 drives became 0.895 on 200; validation of 1.0 hid a real loss; invalid first measurement; non-repeatable traffic. Missing: a resampling study of the selection bias.	Analysis of existing data
2	Where does perception damage start to matter?	Measurement and the null result exist. Missing: a grid of detector loss against traffic density.	About 9 hours per model
3	The teacher–student information gap	14 of 15 failures solved by the collision-aware teacher. Missing: the unevaluated student of that teacher, a heading-aware grid, three training seeds.	1 to 2 days
4	Imitation against reinforcement learning	Six models at 200 drives, four documented failures. Missing: seeds, a stronger reinforcement-learning attempt.	About 2 hours per model
5	Robustness to position error by redundancy	Curves at 40 drives. Missing: 200 drives, a real second cue.	About 11 hours
6	Learned adaptive behaviour: caution from perception reliability	About 35% less estimated energy at 200 drives, success tied at a 160-second limit. Missing: a regime where weather hurts, and a speed-matched control.	About 9 hours
7	Benchmark and dataset release	Everything is stored with hashes. Missing: push to GitHub, licence check, repeatable traffic.	1 to 2 days

Experiments to add first, in order: (i) evaluate the student of the collision-aware teacher and the plain-grid six-number student on the 200 test drives (about 1.5 hours); (ii) two more training seeds for the proposed driver; (iii) the detector-loss against traffic-density grid; (iv) a heading-aware grid.

12 Where to find everything

What	Where (under D:/bev_Fresh/bevcapstone/)
This guide	report/Directors_Research_Guide.pdf
Complete technical report (about 80 pages, every detail, file inventory)	report/Complete_Technical_Research_Report.pdf
All 33 models, mapping of old to new names, hashes	trained_models/ (MODEL_NAME_MAPPING.md, MODEL_MANIFEST.json)
Front-camera videos (scenario 5169)	report/videos/
Figures and the scripts that produce them	report/figures/, report/make_report_figures.py
Code of this project (not on GitHub)	bev_rl/ (results in bev_rl/adaptive/results/)
Teammates' modules (on GitHub)	bev_policy/, bev_state/, camera/, YOLO+weathermodule/, baseline/
Earlier drafts, first version of the report	archive/